Studio Nx

AI 구현 설명 87 — S1-13 템포 변경 (FR-202a)

2026-10-21

AI 구현 문서 87 — S1-13: 템포 변경 (FR-202a)

항목	내용
작업	S1-13 MVP 안정화의 첫 번째 수정: 곡 중간 템포 변경이 저장, 조회, 재생, 재생선, 내보내기에 반영되게 함
작성	Claude (AI), 2026-10-21, 사용자 검토
기반	S1-12 병합(develop, PR #95), DB의 `tempo_changes` 표(2026-10-05)
백엔드	`models/tempo_change.py`, `schemas/tempo.py`, `services/tempo_service.py`(신규), `api/v1/projects.py`, `services/full_project_service.py`, `schemas/full_project.py`, `models/__init__.py`의 변경
프런트	`sequencer-devices.js`, `sequencer-audio.js`, `sequencer-arrange.js`의 변경
함수 문서	백엔드 308 ~ 310, JS 311 (4건)
시험	백엔드 22건 추가(전체 1105건 통과), 화면 시험 20건(`docs/handoff/s1-13_tests/ui_tempo.mjs`)
이전 문서	86_s1-12_audio_regression.md, 84_s1-11b_wav_export.md

1. 처음 생각과 달랐던 점

S1-11b를 끝내며 "알려진 한계"에 "템포 변경 목록을 화면이 아직 넘기지 않는다"고 적었습니다. 이번에 고치려고 살펴보니 문제가 더 컸습니다.

처음 생각	실제
화면이 템포 변경 목록을 내보내기에 안 넘기는 것뿐	목록 자체가 만들어지지 않았습니다. DB 표 `tempo_changes`는 있었지만 읽고 쓰는 모델과 API가 없었고, `GET /full`의 `tempo_changes`는 늘 빈 목록이었습니다.
내보내기만 틀림	재생도 틀렸습니다. 재생 세션이 템포 지도를 받지 못했습니다.
(생각하지 못함)	화면의 재생선과 상단 시간 표시도 시작 템포 하나로 계산해서, 템포가 바뀐 곡에서는 재생선이 소리보다 앞서 달렸습니다.

오디오 계산(`timing.js`, `scheduler.js`, `offline.js`)은 처음부터 템포 변경을 지원했습니다. 빠져 있던 것은 데이터가 서버에서 화면을 거쳐 그 계산에 닿는 길이었습니다.

2. 실행 결과

시험 전용 곡: 오디오 클립(2마디)이 있고, 둘째 마디부터 템포가 절반입니다. 시작 템포는 프로젝트 값(124 BPM)입니다.

확인한 장면	고치기 전	고친 뒤
서버에 변경점 저장과 조회	할 수 없음	`PUT`으로 저장, `GET`과 `GET /full`로 조회
WAV 내보내기 길이	3.87초 (변경 무시)	5.81초 (템포 지도)
내보내기 창의 길이 표시	3.9초 근처	5.8초 근처
재생 엔진의 템포 지도	비어 있음	변경점을 받음
4.2초 경과 시점의 재생선	시작 템포로 계산한 위치(2.17마디)로 앞서 감	소리와 같은 위치(약 1.6마디)
상단 시간 표시	시작 템포로 계산해 실제보다 작음	소리의 경과 시간과 같음

고치기 전 상태는 `sequencer-audio.js`의 연결만 되돌려서 확인했습니다. 그때 같은 시험의 16건 중 6건이 실패했습니다(파일 길이 3.87초, 엔진이 변경점을 못 받음, 재생 위치가 앞섬). 재생선과 시간 표시의 "고치기 전" 값은 코드의 계산식(시작 템포 하나)으로 구한 값이고, 그 부분만 따로 되돌려서 측정하지는 않았습니다.

3. 무엇을 만들었나

```
DB      tempo_changes (이미 있음)          tick, bpm, ramp, UNIQUE(project_id, tick)
백엔드   models/tempo_change.py            TempoChange, TempoRamp                    (308)
        schemas/tempo.py                   요청, 응답 모양과 검증 (422)               (308)
        services/tempo_service.py          목록 읽기, 통째 교체, 권한                 (309)
        api/v1/projects.py                 GET, PUT /projects/{id}/tempo-changes     (310)
        services/full_project_service.py   GET /full 의 tempo_changes 를 채움        (310)
프런트   sequencer-devices.js               변경점을 담고 playbackTempoChanges 로 내줌  (311)
        sequencer-audio.js                 재생 세션과 내보내기 스냅샷에 넘김          (311)
        sequencer-arrange.js               재생선과 시간 표시가 템포 지도를 따름       (311)
```

API	내용
`GET /projects/{id}/tempo-changes`	변경점 목록(tick 오름차순). 멤버면 누구나
`PUT /projects/{id}/tempo-changes`	보낸 목록으로 통째 교체. 빈 목록이면 모두 지움. OWNER와 EDITOR만
`GET /projects/{id}/full`	응답의 `tempo_changes`가 저장된 변경점을 담음

4. 요구사항 대응

요구사항	내용	어디서
FR-202a	곡 중간 템포 변경(`STEP`, `LINEAR`)을 `tempo_changes`에 저장하고, 시간 계산에 반영	308 ~ 311
FR-254	프로젝트 전체 조회(`/full`)에 템포 변경점 포함	310
(이번 범위 밖)	Tempo 줄에서 점을 찍어 편집하는 화면	아래 §7

5. 왜 이렇게 (이번 작업의 결정)

결정	이유
데이터 경로를 끝까지 만들었습니다	화면만 고치면 되는 문제가 아니었습니다. 저장된 변경점이 없으면 재생과 내보내기에 넘길 것이 없습니다.
점 하나씩 고치는 API 대신 통째 교체(`PUT`)입니다	템포 줄은 점 전체가 하나의 곡선이고, 노트 저장(`PUT /patterns/{id}/notes`)과 같은 방식이라 중간 상태가 남지 않습니다.
재생, 재생선, 시간 표시, 내보내기가 같은 함수(`playbackTempoChanges`)를 씁니다	소리와 화면이 같은 템포 지도를 봐야 어긋나지 않습니다. 한 곳만 고치면 다시 어긋납니다.
재생선의 위치를 "시각으로 바꿨다가 다시 위치로" 계산합니다	템포가 바뀌는 지점을 지나는 프레임에서도 오차가 쌓이지 않습니다.
응답의 `bpm`을 숫자로 보냅니다	오디오 엔진이 숫자만 받고 문자열이면 오류를 던집니다 (프로젝트 설정의 `bpm`은 문자열).
박자 변경(`meter_changes`)과 마커는 이번에 하지 않았습니다	같은 종류의 작업이지만 별도로 다룹니다. 칸은 그대로 빈 목록입니다.
Tempo 줄(편집 화면)은 만들지 않았습니다	`sequencer.js`의 `renderGlobalLanes`는 직접 구현하는 템플릿으로 남겨 두었습니다 (프로젝트의 JS 구현 방침). 이번에는 그 화면이 쓸 저장과 소리 쪽을 갖췄습니다.

6. 시험 중에 발견해서 고친 문제

시험 중에 새로 발견된 오류는 없었습니다. 구현 전에 코드를 읽다가 발견한 것이 위 §1의 세 가지입니다.

기존 시험과의 관계도 확인했습니다. 앱의 소리 코드와 오디오 회귀 테스트(S1-12)는 바뀐 것이 없고, 재생선을 고친 `sequencer-arrange.js`를 쓰는 화면 시험 가운데 S1-10(27건), S1-10a(14건), S1-11a(36건), S1-11b(46건), S1-11c(39건)는 모두 그대로 통과했습니다. 한 가지 예외가 S1-09 화면 시험입니다. 이 시험은 두 건이 실패하고 두 번째 실행부터는 멈춥니다. 원인은 이번 변경이 아니라 시험이 오래되었기 때문입니다. 하나는 Connect 칸이 "S1-10 안내"라고 기대하는데 S1-10a에서 Connect가 구현되었고, 다른 하나는 컴프레서 노브(임계값)가 이미 최댓값(0)이라 위로 끌어도 값이 바뀌지 않습니다. 이번 변경을 모두 되돌린 상태에서도 같은 시험이 멈추는 것을 확인했습니다. 이 시험은 이후 Edge 시험(88)에서 현재 화면에 맞게 고쳐 40건이 통과합니다.

시험 방법에서 한 가지 고친 것이 있습니다. 화면 시험 첫 버전은 재생 위치만 확인했는데, 재생선과 시간 표시가 같은 원인으로 틀렸다는 것을 코드를 읽다가 알게 되어 두 가지를 시험에 추가했습니다 (16건에서 20건).

7. 알려진 한계

한계	내용
화면에서 템포를 편집할 수 없습니다	Tempo 줄이 아직 없어서 지금은 API로만 변경점을 만들 수 있습니다. 이것이 이 기능의 남은 큰 부분입니다.
박자 변경은 반영하지 않습니다	재생선과 시간 표시는 4/4로 계산합니다. `meter_changes`는 모델과 API가 없습니다.
Sync 딜레이의 간격은 시작 템포를 씁니다	이펙트가 받는 템포는 곡 전체에서 하나입니다. 곡 중간에 템포가 바뀌어도 간격이 바뀌지 않습니다.
재생선과 소리는 따로 움직입니다	재생선은 화면의 시계로, 소리는 오디오 시계로 움직입니다. 둘이 같은 템포 지도를 쓰지만 시작 시점에 샘플을 불러오는 시간만큼(수십에서 수백 ms) 차이가 날 수 있습니다. 이 구조는 S1-11a부터 그대로입니다.
Chrome만 시험했습니다	Edge에서는 88에서 시험했고 모두 통과했습니다.

8. 남은 일과 다음 작업이 쓰는 곳

항목	내용
S1-13의 남은 일	Sentry 오류 정리, 그 밖의 알려진 한계 정리 (Edge 시험과 S1-09 시험 고치기는 88에서 끝남)
Tempo 줄 구현	`renderGlobalLanes`를 구현하면서 점을 끌 때마다 `PUT /tempo-changes`를 부르면 됩니다. 재생과 내보내기는 이미 그 목록을 읽습니다.
회귀 테스트 (S1-12)	템포 변경이 있는 `full_mix_tempo` 프로젝트가 이미 있습니다. 이 프로젝트는 오프라인 렌더링을 직접 부르므로 이번 수정과 무관하게 계속 통과합니다.

9. 자주 받을 질문 대비 (Q/A)

Q. 템포를 바꾸면 노트의 위치도 바뀌나요? A. 아닙니다. 노트, 클립의 위치는 모두 tick(박 단위)이라서 그대로입니다. 템포 지도가 tick을 실제 시간(초)으로 바꾸는 규칙을 정하므로, 같은 마디가 더 오래 또는 짧게 걸릴 뿐입니다 (DB.md의 템포 지도 설명).

Q. `STEP`과 `LINEAR`는 어떻게 다른가요? A. `STEP`은 그 위치에서 템포가 바로 바뀝니다. `LINEAR`는 직전 변경점부터 그 위치까지 서서히 바뀝니다 (`timing.js`가 로그를 이용해 정확히 적분합니다).

Q. 시작 템포는 어디에 저장되나요? A. `projects.bpm`입니다. `tempo_changes`는 그 뒤의 변경점만 가지므로, tick이 0 이하인 변경점은 만들 수 없습니다 (422).

Q. 내보낸 파일의 길이가 화면과 다르면 어떻게 하나요? A. 이제 같은 템포 지도로 계산하므로 달라지지 않습니다. 시험에서 내보낸 파일이 5.81초이고 템포 지도의 길이와 같은 것을 확인했습니다.